Tabla de contenidos

1. Conceptos básicos de redes de datos (*networking*)
    a. ¿Cómo se comunican los computadores?
    b. ¿Cómo identificamos cada entidad dentro de la Internet?
        i. Dirección IP
        ii. Recordando direcciones IP
        iii. Puertos
    c. Protocolos de comunicación
        i. TCP (*Transmission Control Protocol*)
        ii. UDP (*User Datagram Protocol*)
    d. Encapsulamiento
    e. Arquitectura cliente-servidor

Conceptos básicos de redes de datos (*networking*)

Hasta ahora, nuestros programas pueden leer o escribir archivos, y pueden correr varias tareas de forma pseudo-simultánea. Sin embargo, no hemos visto ninguna manera de interactuar con otros programas. El uso de redes de computadores nos permitirá crear programas capaces de comunicarse con otros programas, sin importar si están corriendo en la misma máquina o en distintas máquinas.

¿Cómo se comunican los computadores?

Para que dos máquinas se puedan comunicar, necesitamos:

1. Un **medio** por donde transmitir los mensajes. Estos medios pueden ser cableados, como los cables telefónicos y los cables Ethernet (llamados típicamente "cables de red"); o también pueden ser medios inalámbricos como el WiFi, las señales de telefonía y datos móviles.
2. Saber con qué máquina queremos comunicarnos: un **destinatario**, y cómo encontrarlo. Para ello usaremos las direcciones IP.
3. Un **protocolo para transmitir mensajes**: un conjunto de reglas que permite a los sistemas lograr que un mensaje emitido por un programa, sea correctamente recibido por otro programa. Veremos dos protocolos: TCP y UDP.
4. Un **protocolo de aplicación**: un conjunto de reglas que todos los programas deben respetar para que la comunicación sea efectiva entre ellos. Estos protocolos definen aspectos como el orden, formato, y contenidos de los mensajes. Protocolos como HTTP, DNS, SMTP, son ejemplos de ellos. Para cada programa definiremos las reglas del protocolo que usaremos.

¿Cómo identificamos cada entidad dentro de la Internet?

Dentro de la red, y en particular dentro de una tan grande como Internet, hay miles de millones de máquinas. Nos referiremos a cada una de estas máquinas como un ***host***. Un *host* es una

máquina que puede ejecutar programas que son capaces de comunicarse con el exterior mediante envío y recepción de datos.

Dirección IP

Todos los *host* que se encuentran en una red reciben una etiqueta llamada **dirección IP**, que sirve para distinguirlos de todos los demás de la red. Este número debe ser único (casi) en toda la red y esto permite que los dispositivos llamados *router* (si tenemos conexión a Internet seguramente tenemos uno de estos dispositivos en nuestro domicilio) puedan **encontrar al destinatario** usando esa dirección IP y hacer que el mensaje llegue al *host* correcto. Naturalmente un desafío es asegurarse que la direcciones IP sean únicas en todo el mundo, pero de eso se encargan los proveedores de Internet (ISP, por *Internet Service Provider*, como Movistar, VTR, Entel, GTD, WOM, etc).

Para comunicarnos con un *host* entonces, necesitamos su dirección IP. En la actualidad, tenemos dos versiones para esta dirección.

- **IPv4:** Es un número de 4 *bytes* (32 bits). Normalmente se escriben en una notación *human-readable*, donde cada *byte* se escribe separado por un punto, como por ejemplo, `192.30.253.112`. Hay $2^{32} = 4.294.967.296$ posibles direcciones en este formato, cantidad que actualmente es insuficiente para la cantidad de dispositivos conectados a internet. Por esto mismo, se desarrolló IPv6 para solucionar este problema.
- **IPv6:** Este tipo de direcciones tiene 128 bits. Se escribe como ocho grupos de 16 bits separados por dos puntos, donde cada grupo de bits se escribe como un número hexadecimal, como por ejemplo, `2001:0db8:85a3:0000:0000:8a2e:0370:7334`. Hay 2^{128} (un número de 39 dígitos)* direcciones posibles en este formato.

**Si te interesa tenerlo, el número es 340.282.366.920.938.463.463.374.607.431.768.211.456*

Recordando direcciones IP

Ahora bien, si estás leyendo esto desde GitHub, notarás que nunca tuviste que ingresar una dirección IP. Lo mismo pasa si vas a `cursos.canvas.uc.cl`. Sin embargo igual puedes recibir los datos de esos *host*. Esto es porque hay otro protocolo que funciona en las sombras que es capaz de hacer automáticamente la traducción desde un **nombre** a una dirección IP, tal como si se trata de una lista de contactos en que buscamos a alguien por su nombre y almacenamos su número telefónico en lugar de recordarlo de memoria.

El protocolo que mantiene este "registro telefónico mundial" se llama **DNS** (*Domain Name Service*), y nos evita tener que recordar direcciones IP. En lugar de ellas usamos **nombres jerárquicos** como `cursos.canvas.uc.cl` que se refiere a un *host* de nombre `cursos` dentro de un **dominio** de nombre `canvas.uc.cl`, el cual a su vez es parte de un **dominio** más grande de nombre `uc.cl`, y que a su vez es parte de un **dominio** más grande aún, denominado `cl` que concentra todas los nombres utilizados en Chile y que es mantenido por la empresa NIC.cl.

Puertos

Dentro de un mismo computador (*host*) se ejecutan varios programas en forma simultánea. Varios de ellos podrían querer comunicarse a través de la red. Por ejemplo, tu navegador web y tu cliente de mensajería favorito. Ahora, ¿te imaginas lo complicado que sería si los mensajes llegaran mezclados? A raíz de esto, cada programa usa un **puerto** distinto para comunicarse.

Esto es como un edificio donde conviven múltiples personas que comparten la misma dirección (dirección IP), pero que se diferencian por su número de departamento (el puerto).

Un puerto es un número de 16 bits, por lo que existen $2^{16} = 65.536$ puertos posibles. Estos números van desde 0 hasta 65.535. Cada aplicación puede ocupar uno o más puertos de un *host*, pero **sólo una aplicación puede estar ocupando un mismo puerto a la vez**. El puerto de origen no tiene que ser igual que el puerto de destino. Por ejemplo, los servidores web usan, por convención, el puerto 80 ó bien el 443 para recibir solicitudes desde navegadores web. No obstante, tu navegador web puede emitir su solicitud desde cualquier otro puerto en tu computador.

Normalmente, los servidores reciben las solicitudes en puertos predefinidos. En la tabla de abajo, hay algunos ejemplos.

Puerto	Descripción servidor
21	FTP
22	SSH
25	SMTP (email)
37	Time
53	Domain Name System (DNS)
80	HTTP (web)
110	POP3 (email)
443	HTTPS (web)

Podemos ver en esta imagen una pared de un edificio donde varios programas comparten la misma dirección IP, pero cada uno utiliza un número de ventana (número de puerto) distinto.

En conclusión, para comunicarnos con un programa que está ejecutando en otro *host*, **necesitamos la dirección IP y el puerto del destinatario**.

Protocolos de comunicación

En la Internet existen dos protocolos que permiten que un mensaje emitido por un **programa emisor** sea recibido por un **programa receptor**. Estos protocolos se conocen como **protocolos de transporte**.

Los mensajes en la Internet son, en el fondo, **secuencias de bytes serializados**. Los mensajes pueden ser muy grandes (imaginemos una película de varios GB) y no se envían por la red como una sola unidad, ya que hay una alta probabilidad que lleguen con errores que se producen tantos en los medios cableados como en los inalámbricos (sobre todo por interferencia electromagnética). Por lo tanto los **protocolos de transporte** dividen el mensaje en unidades más pequeñas llamadas **paquetes**. Cada paquete se etiqueta con la **dirección IP y puerto del destinatario**, tal como cuando mandamos una encomienda, y con su **posición dentro del mensaje original**, de manera que el destinatario, al recibir todos los paquetes, pueda reconstruir el mensaje original.

Si bien los paquetes son suficientemente pequeños para que haya menor probabilidad de errores, siempre existe la posibilidad de que algún paquete llegue a su destino con algunos *bytes* cambiados, o peor aún que no llegue y se pierda. La Internet es muy grande y en algunos momentos hay congestión en algunos puntos y eso puede provocar que los *routers* "boten" algunos paquetes. Los protocolos de transporte pueden actuar de manera distinta frente a esos casos.

TCP (*Transmission Control Protocol*)

El protocolo TCP sirve para enviar y recibir un *stream* ("corriente" o "flujo") de *bytes serializados* en forma **confiable** a un destinatario. La palabra "confiable" hace referencia a que se asegura que **todos los paquetes llegan de manera íntegra** sin errores. Esto lo consigue mediante (1) funciones que se aplican a cada paquete para verificar que no fueron alterados; (2) solicitando al receptor que confirme la llegada íntegra de cada paquete; y (3) retransmitiendo aquellos que se han perdido o que han llegado dañados. A medida que los paquetes van llegando correctamente y en orden, la aplicación receptora puede comenzar a leer los datos, sin necesidad de esperar el último paquete del mensaje.

Antes de iniciar una transmisión usando el protocolo TCP, ambas partes, emisor y receptor, deben ejecutar un protocolo de "establecimiento de conexión" conocido como ***handshake*** (saludo de manos), durante el cual se ponen de acuerdo en parámetros de la conexión como tiempo de espera, cantidad de paquetes simultáneos, uso de memoria, etc. Es similar a cuando efectuamos una llamada telefónica en que el emisor debe conectarse con el receptor antes de empezar a comunicarse datos. Por esto se dice que TCP es un protocolo ***connection-oriented*** (orientado a la conexión).

Todo este proceso que hace TCP es para garantizar la **fiabilidad** de la transmisión. Sin duda, esto implica un cierto tiempo de demora (denominado "latencia") en la transmisión de datos; sin embargo, en muchas aplicaciones, por ejemplo cuando subimos nuestro código a GitHub, es mucho más importante que los datos lleguen de manera íntegra y correcta a que lleguen rápido. No nos gustaría que nuestras tareas fallaran porque la red se equivocó en transmitir unos pocos *bytes*. La verdad es que tanto el cable como la comunicación inalámbrica cometen bastantes errores (y entre ellos, el inalámbrico posee mucho más errores que el cableado), pero afortunadamente el protocolo TCP está ahí para protegernos.

Otros protocolos de aplicación como HTTP (el de las páginas web), SMTP (para enviar correo electrónico), BitTorrent (para transferencia de archivos), usan a TCP para asegurarse que las transmisiones no tengan errores.

En los contenidos de la otra semana, veremos cómo implementar programas en *Python* que usen directamente el protocolo TCP para comunicarse entre sí.

UDP (*User Datagram Protocol*)

El protocolo UDP sirve para enviar y recibir paquetes (datagramas), pero **sin garantías de envío íntegro**. Algunos paquetes podrían perderse, y UDP no toma ninguna acción al respecto. Si un paquete llega con errores, UDP los descarta y no hace esfuerzos por volver a solicitarlos. Se dice que UDP hace "su mejor esfuerzo" (*best-effort*) para enviar los mensajes, pero no entrega certezas.

UDP es un protocolo **connectionless** (sin conexión), lo que significa que los mensajes se envían sin que emisor y receptor se pongan necesariamente de acuerdo en comunicarse. Solo necesita que el receptor este esperando comunicación de "alguien". No hay un *handshake*. Los paquetes simplemente llegan (o no).

UDP tiene la ventaja de ser un protocolo mucho más liviano que TCP (tiene menor sobrecarga, u ***overhead***) al saltarse el *handshake*, evitar confirmaciones y retransmisiones. Posee menor latencia que TCP. Pero, ¿de qué nos sirve un protocolo que no garantiza que los datos llegan íntegros?. Aplicaciones donde se privilegia la velocidad de transmisión que la integridad de los datos son la transmisión de audio y video. En estos casos la pérdida de algunos pocos *bytes* solo se notará en que el video pierda algún cuadro (se ve "pixelado") o que el audio tenga un chasquido, pero mientras no sean muchos *bytes*, la comunicación fluye más rápidamente.

En este curso no abordaremos el protocolo UDP en profundidad más allá de su concepto.

Encapsulamiento

La comunicación entre dos dispositivos o procesos se divide conceptualmente en capas. Cada capa tiene una responsabilidad, usa la anterior y confía en que esta funciona correctamente. Existen dos modelos de capas, equivalentes entre sí: uno de ellos es el modelo OSI y el otro es el modelo TCP/IP.

Nuestros programas en Python utilizarán el modelo TCP/IP. Funcionarán **siempre en la capa de aplicación**, y enviarán y recibirán mensajes **utilizando un protocolo de transporte** (TCP ó UDP), el cual ya está provisto por el sistema operativo.

Arquitectura cliente-servidor

Finalmente, para poder implementar códigos en Python que permitan comunicar distintos dispositivos por la red, es necesario comprender la **arquitectura cliente-servidor**.

Una **arquitectura** en computación es una manera de conectar o construir *hardware* o *software* para que cumpla ciertos estándares. La **arquitectura cliente-servidor** es una manera de conectar dos o más programas donde uno cumple el **rol de servidor**, el cual **ofrece servicios** a otros programas que poseen el **rol de cliente**. Los programas cliente **efectúan solicitudes al servidor** y **consumen sus servicios**.

En esta arquitectura el **cliente** es quien **inicia la conexión**, y establece una conexión con el **servidor** de acuerdo a algún protocolo. Por otro lado, el **servidor**, al iniciarse, se dedica a esperar y **escuchar conexiones entrantes** de clientes, y reacciona a cada una de ellas. Al recibir una solicitud de conexión, el servidor puede decidir si la acepta o no, efectúa una acción predefinida, y envía una respuesta al cliente según lo diga el protocolo de comunicación. La transmisión puede hacerse vía TCP ó UDP.

Fuente: http://en.wikipedia.org/wiki/Client%E2%80%93server_model Fuente de la imagen: Wikipedia

---

NOTA: Esta semana utilizaremos principalmente los conceptos de puertos y direcciones IP, la próxima semana profundizaremos más en el uso de protocolos y el encapsulamiento.